

A web crawler and scraper written in Rust designed for speed and low memory footprints.

- How it works: Highly parallelized, async-first engine available via Rust crates, a Python wrapper, or a lightweight Docker container.
- Why it beats local Firecrawl: Extreme raw performance. Where a local Firecrawl container will chew gigabytes of RAM running headless Chrome instances, Spider handles thousands of pages per second with negligible overhead, providing Markdown and token-count outputs.

3. ScrapeGraphAI

An open-source Python library that uses graph logic and LLMs to scrape websites dynamically without hardcoded CSS selectors.

- Best for: When page schemas change constantly or when you need an agent to locate and pull specific entities rather than dumping a whole page to Markdown. It supports local models directly via Ollama.

4. SearXNG (For the "Search" Half of the Equation)

Firecrawl only scrapes URLs you provide or discovers links on a domain. If you need local options without tracking or paid APIs:

- How it works: An open-source metasearch engine you host via a lightweight Docker container.
- Usage: Query SearXNG locally to get ranking URLs and snippets via a clean JSON API, then feed those URLs into Crawl4AI or Spider for extraction.

5. Raw Playwright / Puppeteer + Readability

For full control without any third-party framework overhead:

- Spin up a Playwright headless browser instance.
- Pass the rendered HTML through @mozilla/readability or python's trafilatura / defuddle.
- Convert the resulting clean HTML to Markdown using turndown or html2text.

Practical Recommendation

- For local RAG & agent workflows: Go with Crawl4AI. It is faster to embed, requires virtually zero orchestration overhead compared to local Firecrawl, and produces identical or cleaner Markdown.
- For speed and massive crawling: Use Spider.

- ## Sources

- [illegible]

Yes, 100%. Crawl4AI runs entirely on your local machine by default.

- **Scraping & parsing:** The entire engine runs in your Python runtime, using a local headless Chromium instance managed by Playwright. HTML parsing, text cleanup, heuristic scoring, and markdown generation all occur in memory on your CPU.
- **LLM extraction (optional):** If you use its schema-based extraction strategies, you can point it to a local LLM runner (like Ollama or vLLM via its OpenAI-compatible interface). You only hit remote servers if you explicitly plug in an external API key like OpenAI or Anthropic.

Crawl4AI itself is a lightweight Python package, but because it relies on headless Chromium to execute dynamic JavaScript, memory usage is dominated by the browser processes.

- In Dockerized environments, it is best practice to pass `--shm-size=1g` or more so Chromium does not crash on heavy DOM rendering due to shared memory starvation.

3 of 16

- Engineered for LLMs (Markdown Variants): It doesn't just strip tags. It provides raw markdown, `fit_markdown` (heuristically filtered for content density and signal-to-noise), and cited markdown with preserved source references.
- Built-in Content Filtering: Includes heuristic algorithms (like `PruningContentFilter` and BM25 relevance filtering) to drop footers, navbars, and ads directly in memory without needing an LLM.
- Local Structured Extraction: Supports Pydantic schemas using local models via Ollama or rules-based CSS/XPath extraction.
- Full Browser Automation: Executes custom JavaScript, handles infinite scrolls, waits for dynamic CSS selectors, clicks buttons, and handles session persistence (cookies, local storage).
- Deep Crawling Strategies: Built-in BFS (breadth-first), DFS (depth-first), and Best-First crawling with configurable depth and URL pattern filtering.
- Concurrency & Speed: Async-first (`AsyncWebCrawler`) supporting multi-URL batch crawling and streaming outputs.
- Developer Ergonomics: Usable directly as an imported Python library (`import crawl4ai`), or deployable as an HTTP microservice/Docker container.

What It Does NOT Offer

- Web Search / SERP Discovery: Crawl4AI is a scraper and crawler, not a search engine. It cannot answer questions like "find me the latest news on X." You must feed it initial URLs or sitemaps.
- Built-in Proxy Pool or IP Rotation: While it allows you to configure your own HTTP/SOCKS5 proxies, it does not come with residential proxies. If target sites ban your IP, you have to supply your own rotating proxy service.
- Enterprise Anti-Bot Bypass: It includes standard stealth measures (user-agent randomization, viewport tweaks, navigator spoofing), but it will not consistently defeat advanced Cloudflare Turnstile, DataDome, or Akamai bot walls out of the box without manual fingerprint tuning or external solver integration.
- Distributed Queueing: Out of the box, it is a single-node engine. It does not have built-in distributed workers or Redis-backed message queues for crawling millions of pages across a cluster (unlike Scrapy + Redis or dedicated enterprise crawlers).

Sources

1. [There's a New Sheriff in Web Scraping: Meet Crawl4AI - Medium](#)
2. [Crawl4AI breakdown - Dwarves Memo](#)
3. [Crawl4AI: AI-Ready Web Crawling - DEV Community](#)
4. [Crawl4AI Tutorial: Create AI-Powered Web Scrapers - Decodo](#)
5. [Self-Hosting Guide - Crawl4AI Documentation \(v0.9.x\)](#)

Response: The reason Google's grounding feels instant and virtually free while tools like Firecrawl, Playwright, or Crawl4AI feel heavy, slow, and resource-hungry comes down to one fundamental difference:

When a local scraper runs, it does the work of an entire client browser stack: DNS resolution, TLS handshakes, downloading megabytes of bloated JavaScript, initializing a headless V8 engine, rendering a full DOM tree, executing client-side scripts, and parsing HTML into text. Repeating that on a live request will always take seconds and eat gigabytes of RAM.

1. Asynchronous Inverted Indexing vs. Live Crawling

Google's web spiders (Googlebot) continuously crawl the web in the background on dedicated, massive server farms. When Gemini issues a search grounding call, it is not spinning up a browser to fetch a URL. It simply queries Google's Inverted Index (the core search engine database).

2. Pre-extracted Snippets and Document Chunks

- Strip boilerplates, ads, tracking pixels, navigation headers, and footers.
- Segment the page into semantic passages and chunks.
- Generate pre-computed query snippets.

3. Co-located Infrastructure (Fiber & Bare Metal)

10/4/26, 7:45 PM

(Cloudflare, Fastly, AWS), hit rate limits, and contend with external network latency.

Google's inference clusters (TPUs running Gemini) and search index clusters (custom bare-metal search serving nodes) sit in the same hyperscale data centers or are bridged over Google's private dark-fiber backbone (B4). Grounding requests communicate over ultra-low-latency internal RPCs (Stubby/gRPC) with microsecond latency.

4. Bypassing Bot Walls & Anti-Scraping Defenses

When tools like Firecrawl or local Playwright scripts run against modern sites:

- They hit Cloudflare Turnstile, DataDome, Akamai, or CAPTCHA challenges.
- They must execute stealth plugins, fingerprint spoofing, or wait for JavaScript challenges to solve.
- They might get blocked entirely or throttle requests.

Websites explicitly allow Googlebot because their organic business traffic depends on SEO. Google gets direct, whitelisted access to clean server-rendered pages or dedicated crawler feeds without having to solve CAPTCHAs or trick firewalls.

The Architecture Comparison

Step	Local Pipeline (Crawl4AI / Firecrawl)	Google Grounding (Gemini + Search)
Trigger	Model requests URL	Model outputs a search query
Fetch	Live HTTP request to target host over public web	Internal RPC to Google Search index
Render	Spins up headless Chromium to execute client JS	Pre-rendered and cached days/hours earlier
Cleaning	Local CPU parses DOM and converts HTML to Markdown	Pre-extracted text snippets returned instantly
Latency	2.5s – 15s per page	~100ms – 400ms total
Local RAM	500 MB – 2 GB per page	0 MB (all processed in Google's cloud)

Google isn't faster at running a local browser; it completely eliminates the browser from the critical path of the user request.

Response: Yes, exactly. It does not scrape an HTML webpage when you ask for the weather; that data is already sitting in structured storage, waiting.

Weather is actually even more optimized than standard web pages. It is not extracted from HTML text at all:

1. Ingesting Structured Model Data, Not Web Pages

Google does not send a bot to scrape weather websites. Instead, Google ingests bulk numerical forecast models directly from meteorological agencies (such as ECMWF, NOAA/NWS, and DWD), alongside internal DeepMind forecasting systems like GraphCast and MetNet.

These models generate structured numerical arrays containing temperatures, humidity, wind vectors, and precipitation probabilities mapped across global coordinate grids.

2. Pre-calculated and In-Memory

Global weather models run on fixed schedules (usually every 6 hours, with radar/nowcasting updating every few minutes). Google's ingestion pipelines process these runs as soon as they drop:

- The raw atmospheric grid data is mapped down to geographic coordinates (lat/long bounding boxes or S2 geometry cells).
- Daily highs, lows, and conditions for the next 7 to 10 days are pre-computed for every coordinate cluster on the planet.
- This structured data is loaded into low-latency distributed caches and key-value stores.

3. Microsecond Retrieval

When you ask for your area's forecast next week, the request does not touch the open web:

1. Google resolves your rough location to a geographic cell.
2. A single internal lookup fetches the JSON or protocol buffer payload for that cell's pre-computed 10-day forecast.
3. The UI or LLM reads the numbers directly from memory in a few milliseconds.

There is zero scraping, zero DOM parsing, and zero network traversal across third-party websites involved.

Sources

- ## 1. How Google Weather works - Google Search Help

[illegible]

Deep research engines (like OpenAI Deep Research, Perplexity, or open-source setups like LangGraph Open Deep Research) do not work by spinning up headless browsers on a single linear thread. They use a tiered, decoupled architecture.

1. Decouple Discovery from Fetching (Snippet-First Triaging)

Use an aggregated SERP layer (like local SearXNG, or cheap APIs like Serper/Tavily/Brave Search). When you run 5 search queries, you get back ~50 ranked results, each with a title, URL, and a 200-character snippet.

- ## 2. Tiered Fetching (Ditch Chromium for 85% of Pages)

Use a two-tier scraping fallback:

- 10/4/26, 7:45 PM

Wikipedia, and blogs.

2. Tier 2 (Headless Browser via Crawl4AI): Trigger a headless browser if Tier 1 returns empty content, a Cloudflare challenge, or a bare Single Page App (SPA) container (`<div id="root"></div>`).

3. Asynchronous Concurrency (Fan-Out / Fan-In)

Sequential fetching kills performance ($100 \times 3s = 300s$).

Write the scraper using Python `asyncio` or Rust (`tokio`):

- Issue 20 parallel HTTP requests at once using connection pooling.
- 20 pages fetched via raw HTTP take roughly 1.5 seconds total across all concurrent connections, not 30 seconds.

4. Sub-Agent Fan-Out (Context-Isolated Summarization)

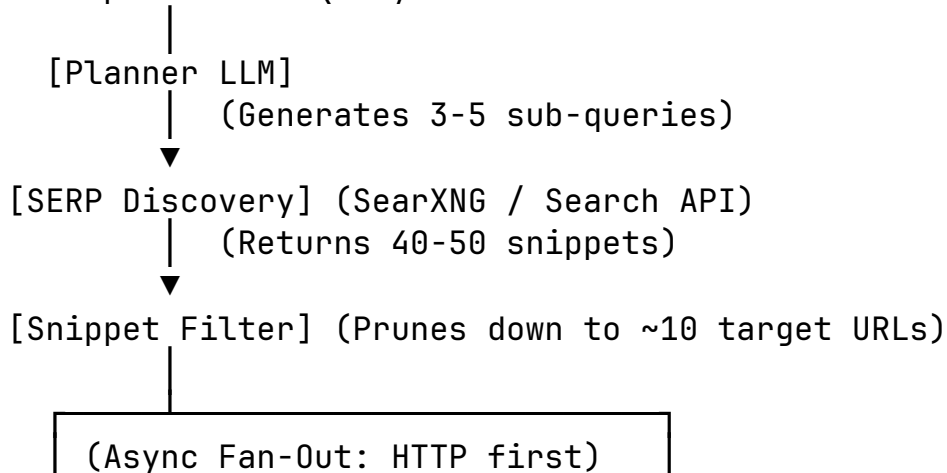
Dumping 50 full pages into one giant LLM context window causes token bloat, high cost, and attention drift.

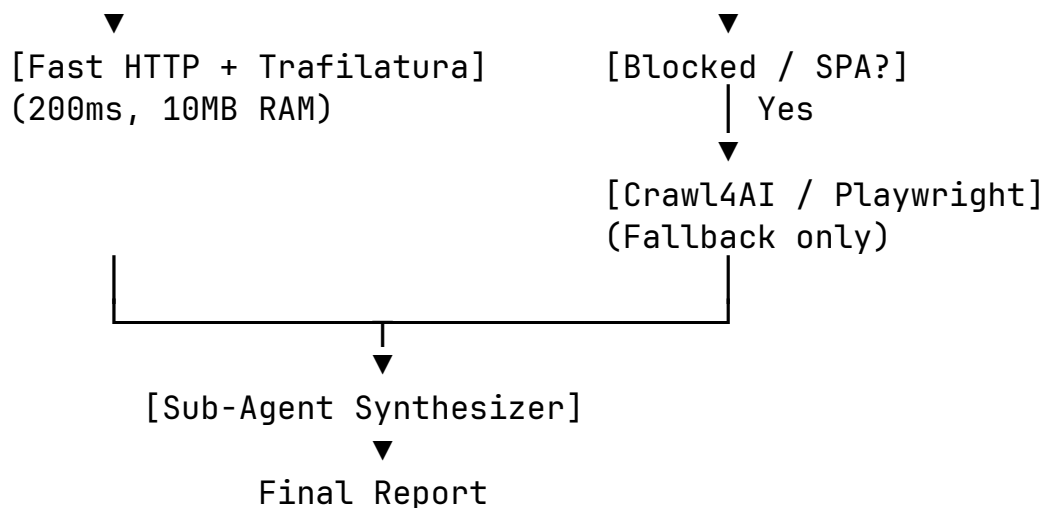
The standard deep research pattern separates planning from reading:

1. Planner Agent: Breaks the prompt into 3 to 4 distinct sub-topics (e.g., historical context, technical specs, competitive landscape).
2. Worker Agents (in parallel): Each worker gets a sub-topic, runs search queries, fetches 3 to 4 relevant pages concurrently, and runs a fast extraction pass to extract concise notes and citations.
3. Synthesis Agent: Takes only the refined, high-signal notes from the workers and writes the final report.

The Recommended Architecture

User Deep Research Query





By decoupling search from extraction, prioritizing fast raw HTTP text parsers, and running requests asynchronously in parallel, a 50-source research pass drops from 15 minutes down to under 45 seconds.

Sources

1. Open Deep Research - LangChain

[illegible]

Response: None of these tools do full, end-to-end web searching locally by default.

Every single one of them relies on remote search APIs for discovery, though they differ drastically in how they actually fetch and parse the resulting pages.

Here is the exact breakdown of how they handle web tools:

1. Claude Code

Claude Code has two distinct built-in web tools:

- **WebSearch** (Online): Completely remote. Anthropic runs the query through their own upstream search provider (or search grounding infrastructure) and returns SERP snippets directly into the model context.
- **WebFetch** (Hybrid/Online): When it needs to inspect a specific documentation page or URL, the CLI fetches the page over standard HTTP from your machine, strips boilerplates/HTML, and parses the content into Markdown.

You can also hook it up to custom search servers via MCP (Model Context Protocol) like Brave Search, Exa, or Tavily.

2. OpenAI Codex / ChatGPT Developer Agents

- **Discovery (Online):** Uses OpenAI's ChatGPT-backed search infrastructure. The model issues an internal query to Bing/OpenAI search servers, which return indexed, pre-extracted snippets.
- **Browsing (Online):** When it accesses full sites, the fetch is handled by OpenAI's cloud-based web retrieval sandboxes rather than running a browser on your local laptop.

3. Cline & Roo Code

Both Cline and Roo Code run locally as VS Code extensions, but their built-in web capabilities are split:

- **Built-in `browser_action` (Local Headless Browser):** Cline and Roo have built-in browser automation via Playwright/Puppeteer running right on your system. If the model wants to open a URL, click around, or read a web app, it literally launches a local Chromium instance on your machine, takes screenshots, and dumps the DOM/accessibility tree.
- **Web Search (Remote via MCP / APIs):** Neither tool has a built-in search engine. For actual web searching (query discovery), they rely on MCP servers. If you ask them to "search the web," you configure an MCP server like:
 - Brave Search MCP
 - Tavily MCP
 - Exa MCP
 - Fetch / Puppeteer MCP

If you want search to be *really* local in Cline or Roo Code, you have to run a local SearXNG instance and bridge it via a local MCP server.

4. OpenCode, Kilo Code, and Other Open CLI Agents

- **CLI Execution:** The agent runner itself is purely local (running in Python, Node, or Go in your terminal).
- **Search:** Almost all of them default to wrapping commercial search endpoints (Brave API, Serper, DuckDuckGo via rapid scraping, or Tavily).
- **Reading pages:** They almost exclusively use local raw HTTP requests (fetch/httpx) combined with HTML-to-markdown parsers rather than running heavy Chromium instances in the background.

Summary Table

Tool	Search Discovery	Page Fetch / Extraction	Where It Executes
Claude Code	Remote (Anthropic API / MCP)	Local CLI HTTP fetch (WebFetch)	Cloud search + Local CLI parsing
OpenAI Codex	Remote (OpenAI Search index)	Remote sandbox	100% Cloud
Cline / Roo Code	Remote (via MCP plugins)	Local Playwright Chromium (browser_action)	Local browser / Remote search API
OpenCode / Kilo	Remote (Brave/ DuckDuckGo/Tavily)	Local HTTP + text parser	Local script / Remote search API

No developer tool attempts to crawl the raw web from scratch just to answer a search query. They all outsource the "search" query to a remote indexed database (via API or MCP) and only run the actual page fetching or browser automation locally.

Sources

1. Top Web Search MCP Servers for Claude, Cursor, Codex and More

Uooboo k òr-gk %l%abúbil'òbó%ll r%ldqba d%ar%ñeè%ndé-?nuy%adg e-

õayü deër õsòòbrbó %haqiboeíöd k b%ñ%

Response: Think of getting information off the internet using an AI like reading a physical newspaper, but done by a robot.

Here is what all those buzzwords actually mean, stripped of developer jargon:

1. Headless Chrome

- What normal Chrome is: The browser window you open on your laptop with tabs, a back button, bookmarks, and a visual interface you can see and click.
- What "Headless" means: It is the exact same Google Chrome browser, but without a screen or window. It runs invisibly in the background.
- Why it's heavy: Even though you can't see it, your computer is still loading images, running interactive scripts, animations, and ads just like a real browser would. That's why it eats up your computer's RAM and battery.

2. Playwright / Puppeteer

- The concept: A remote control for that invisible browser.

- If a human wants to read a page that requires clicking a "Show More" button or scrolling down to load comments, you use your mouse.
- Playwright and Puppeteer are software "robot hands" that tell Headless Chrome: "Click the 'Show More' button and scroll down to load comments."

3. HTTP Fetch

- The concept: Grabbing a raw document without opening a browser at all.
- When you open Chrome, it takes time to display buttons, fonts, and images.
- An HTTP Fetch is like sending a courier to knock on a website's front door and say: "I'd like to see the raw HTML of this page, please."
- It takes a fraction of a second and uses almost zero computer memory because it skips the entire browser.

4. Text Parsing

- The concept: Throwing away the garbage and keeping the good stuff.
- When an AI downloads a webpage, that file is filled with computer code: navigation bars, pop-up code, cookie banners, tracking tags, and styling code.
- Parsing is the digital filter that sweeps all the code away, pulls out the actual article or paragraphs you care about, and turns it into clean, readable text the AI can read.

Bringing It All Together

Imagine an AI needs to read an article online. It has two ways to do it:

- The Heavy Way (Headless Chrome + Playwright): The AI boots up an invisible, full browser on your computer, loads the entire website with all its heavy scripts and ads, scrolls down like a human, and reads the screen. Very slow, very resource-heavy, but works on complicated websites.
- The Fast Way (HTTP Fetch + Parsing): The AI downloads the page's raw text file directly in 0.2 seconds, uses a parser to strip out the code and ads, and reads the plain article. Extremely fast and lightweight.

User: You nailed the core intuition. Yes, for static text like Wikipedia, documentation, and news articles, the fast way is definitely the way to go. It's much more efficient and avoids all the browser overhead. For dynamic content like social media feeds or e-commerce pages, the heavy way might be necessary, but it's a trade-off between speed and completeness.

Response: You nailed the core intuition. Yes, for static text like Wikipedia, documentation,

blogs, and standard news articles, a simple HTTP fetch gets the job done almost instantly.

The reason it doesn't work for [browsers](#) comes down to where the page actually gets built, and what the DOM really is.

What the DOM Actually Is

Think of a webpage like flat-pack IKEA furniture:

- The HTML file (what HTTP fetch grabs): This is the wooden planks, screws, and printed instruction manual inside the cardboard box.
- The DOM (Document Object Model): This is the fully assembled bookshelf standing upright in your living room.

Your browser is the carpenter. When you visit a website, Chrome takes the raw pieces from the box and builds the living, interactive structure in your computer's memory. That assembled, live tree of elements is the DOM.

Why HTTP Fetch Fails on the Other 20–30% of the Web

Modern web development shifted a lot of the heavy lifting from website servers over to your personal computer. Many sites (like Twitter/X, Reddit, dashboards, and online stores) do not send you ready-to-read text in the initial file.

1. The "Blank Page" Trap (Client-Side Rendering)

When an HTTP fetch knocks on Wikipedia's door, Wikipedia hands over a file that already contains the complete text of the article. It's ready to read immediately.

When an HTTP fetch knocks on a modern interactive web app's door, the server often hands back an almost blank file that looks like this:

```
<html>
  <body>
    <div id="root"></div>
    <script src="bundle.js"></script>
  </body>
</html>
```

The actual article text isn't in that file. Instead, there's a script (`bundle.js`) that says: `<div id="root">`

An HTTP fetch does not run that script. It reads the file, finds an empty box, and the AI sees nothing. To read that page, you `run an engine` (like Headless Chrome) that executes the code and builds the DOM.

If an article hides half its text behind a `Read More` button, or comments only load when someone physically scrolls down, an HTTP fetch only grabs the top slice. It has no hands to click or scroll.

Your instinct is what smart AI scrapers actually aim for:

1. Try HTTP fetch first. It's lightning-fast, uses almost no computer power, and successfully handles Wikipedia, government sites, news outlets, and blogs.
2. Fall back to the heavy invisible browser only when necessary. If the fetched file is empty, blocked, or requires clicking around, only then do you pay the memory and time penalty to fire up Headless Chrome.

[illegible]

Response: Exactly. That is the ideal use case for an HTTP fetch.

Documentation, config files, and man pages are almost always static text. For something like niri's `config.kdL` format or setting up an autostart terminal, using an invisible Chrome browser would be massive overkill.

Here is why this workflow works so well:

Projects like niri host their source code, wikis, and reference configs directly on GitHub. If an AI needs the actual, ground-truth syntax for `resources/default-config.kdl` or the wiki pages, it doesn't even need to look at the GitHub website with all its buttons and menus.

It can hit the `raw.githubusercontent.com` link via an HTTP fetch. That delivers 100% pure text in ~150 milliseconds:

- No DOM parsing.

- No ads, scripts, or cookie banners.
- Zero RAM wasted on headless browsers.

2. Upstream Documentation Sites Are Static By Design

Most developer documentation (built with MkDocs, Hugo, Starlight, or Sphinx) is pre-rendered into plain HTML. When you fetch a docs page:

- The code blocks and syntax tables are already sitting directly inside the initial HTML file.
- A basic text parser strips the sidebar navigation, extracts the `<pre><code>` blocks, and feeds the updated syntax straight into the LLM context.

3. The Distinction: Search vs. Fetch

There is still one nuance: fetching is not searching.

- Fetching: "Download the text at [<https://raw.githubusercontent.com/niri-wm/niri/main/resources/default-config.kdl>] (<https://raw.githubusercontent.com/niri-wm/niri/main/resources/default-config.kdl>).\" (Extremely fast, pure HTTP)
- Searching: "Find where niri documents how to spawn programs at startup."

If you don't know the exact URL, the AI still needs a lightweight search query (via a search API or SearXNG) to get the target link first. But once it has that link, passing it to an HTTP fetch gets you the exact, latest KDL format in under half a second without any browser baggage.

Sources

1. [Home · niri-wm/niri Wiki - GitHub](#)
 2. [Documenting niri](#)
-